

The Kerekes Handshake™

Evidence-Backed Resume Protocol Whitepaper & Full Specification v1.5

Author: Jeffrey Kerekes — Systems Practitioner

Why I Built This

I hate writing resumes. After 15+ years self-employed, I realized that modern AI resume tools force everything into a fluffy two-page limit, and every bullet still feels like *"tell, don't show."*

I created the Kerekes Handshake: a dead-simple way to link every claim on your resume to real evidence. Now recruiters or AI can audit you in one click instead of taking your word for it. This won't replace digital wallets for driver's licenses and diplomas — but what about the things no third party would ever certify? Solo audits. Civic interventions. DIY builds. Local wins. That's the gap this fills.

Demo note: This specification uses live proof from a real career. For privacy when you copy it, redact sensitive details in your own version. Real data beats synthetic every time.

Abstract

Traditional resume: Claim → Trust

Kerekes Handshake: Claim → Evidence → Verification

The Kerekes Handshake™ transforms resumes from narrative claims into verifiable, evidence-backed systems. It establishes a structured relationship between professional claims and primary artifacts, enabling both humans and AI systems to audit credibility.

1. Core Concept

Every bullet on a traditional resume is an unverified assertion. The Kerekes Handshake replaces the implicit trust chain with an explicit evidence chain. The protocol is not a credential issuer — it is a structured way to surface evidence you already have.

- No blockchain required.
- No third-party issuer required.

- No wallet required — yet. (See Section 9 on future-proofing.)

2. System Architecture — The Three-Layer Model

A compliant Handshake implementation operates across three layers:

- **Layer 1 — Narrative (Resume):** The human-readable interface. Professional claims are tagged with data-kcm attributes in the HTML source.
- **Layer 2 — Registry (Index):** The `claims.json` ledger. Maps each claim ID to one or more absolute evidence URIs.
- **Layer 3 — Integrity (The Seal):** The `site_manifest.json.asc` — a PGP-signed manifest containing SHA-256 hashes of all artifacts. This is the cryptographic Root of Trust.

Resume (data-kcm) → `claims.json` → `site_manifest.json.asc` → `/evidence/`

Flow: AI agent reads resume → detects KCM tags → loads registry → verifies hashes → retrieves artifacts → returns verdict.

3. File Structure

Place these files at your website root:

<code>/index.html</code>	← The Resume (tagged with data-kcm)
<code>/claims.json</code>	← The Map (claim IDs → evidence URIs)
<code>/site_manifest.json.asc</code>	← The PGP-Signed Root of Trust
<code>/query/</code>	← The AI Audit Portal
<code>/evidence/</code>	← The Vault: PDFs + matching .txt sidecars
<code>/openapi.yaml</code>	← Machine-readable API instructions
<code>/llms-full.txt</code>	← High-density LLM pre-flight context
<code>/.well-known/ai-plugin.json</code>	← Agent discovery manifest

4. Step-by-Step Implementation

This section is written for non-developers. You do not need to write code to implement the core protocol.

Step 1 — Collect & Upload Your Evidence

Create a folder called `/evidence/` on your website. Upload your real primary-source documents: inspection reports, FOI rulings, audit spreadsheets, certificates, anything that verifies a claim you make.

Aim for documents you didn't create yourself — third-party records carry more weight than self-authored ones.

Step 2 — Generate the Text Bridge (v1.5 Requirement)

Why this matters: Most AI crawlers cannot reliably parse binary PDF streams in real-time. They stall, skip, or hallucinate. The Text Bridge solves this.

For every PDF in /evidence/, create a matching .txt sidecar with the identical filename:

```
budget_audit_2011.pdf
budget_audit_2011.txt ← same name, .txt extension
```

Automatic generation (requires poppler-utils):

```
for f in *.pdf; do pdftotext "$f" "${f%.pdf}.txt"; done
```

Manual method: Open each PDF, select all text, paste into a plain text file, save with the same filename and a .txt extension.

AI auditors are instructed to fail-over to the .txt bridge if the binary PDF is unreadable. This ensures accurate extraction of dates, numbers, and names without hallucination.

Step 3 — Create claims.json

Create a file named `claims.json` at your site root. This is the map connecting each resume claim to its evidence files. See Section 6 for the full field specification.

```
[
  {
    "claim_id": "budget_audit_2011",
    "title": "Municipal Budget Reconstruction",
    "description": "Reconstructed city budget using raw FOI data.",
    "domain": "Civic Systems",
    "evidence": [
      "https://yourdomain.com/evidence/budget_audit_2011.pdf",
      "https://yourdomain.com/evidence/budget_audit_2011.txt"
    ],
    "outcome": "Identified $38k/household hidden debt liability"
  }
]
```

Step 4 — Sign Your Manifest (PGP Integrity Seal)

Generate SHA-256 hashes for all evidence files and store them in `site_manifest.json`. Then sign it with your PGP key:

```
gpg --clearsign site_manifest.json
# Produces: site_manifest.json.asc
```

The resulting .asc file is your cryptographic seal. It proves the evidence hasn't been tampered with since you signed it. **Re-sign whenever you add or modify evidence files.**

Verification command (for auditors):

```
curl -s https://yourdomain.com/site_manifest.json.asc | gpg --verify
```

Don't have a PGP key yet? See keybase.io or pgp4win.org for setup guides. This step is required for v1.5 compliance but can be added after initial deployment.

Step 5 — Tag Your Resume with KCM

Open your resume HTML and wrap each professional claim in a data-kcm tag. This is the semantic hook that lets AI agents connect your narrative to the evidence registry.

```
<article data-kcm="budget_audit_2011"
  aria-label="KCM Claim: budget_audit_2011">
  <h3>Municipal Budget Reconstruction</h3>
  <p>Reconstructed municipal budget using raw data.</p>
  <ul class="evidence">
    <li><a href="/evidence/budget_audit_2011.pdf">Audit Spreadsheet</a></li>
  </ul>
</article>
```

Step 6 — Add the Audit Link to Your Resume

Add this line to the top of your resume HTML and PDF:

```
Audit this resume with AI → https://yourdomain.com/query
```

This single line is the entry point for any recruiter or AI agent. It signals that verification is invited, not just possible.

5. KCM — Kerekes Claim Markup

KCM is a lightweight semantic HTML micro-layer that tethers narrative bullets to machine-readable evidence. It requires no JavaScript and no external library.

Required Elements

- `data-kcm="[claim_id]"` — Placed on the parent `<article>` or `<section>` of a claim. The value must match a `claim_id` in `claims.json`.

Optional Enhancement Attributes

- `class="ai-hook"` — Marks specific keywords, case numbers, or forensic pivots within claim text (e.g., FIC 2009-014, \$38,000 per household).
- `itemprop="evidenceOrigin"` — Applied to evidence links to denote primary source artifacts for Schema.org structured data parsers.

AI Verification Logic

An AI agent is considered compliant when it:

- Detects `data-kcm` attributes in the resume HTML.
- Matches each ID against the corresponding entry in `claims.json`.
- Retrieves the linked artifact — using the `.txt` sidecar if the PDF is binary-locked.
- Validates the artifact's SHA-256 hash against `site_manifest.json.asc`.

6. claims.json — Field Specification

Field	Required	Description
<code>claim_id</code>	✓	Unique slug matching the <code>data-kcm</code> value (e.g., <code>budget_audit_2011</code>)
<code>title</code>	✓	Short human-readable label for the claim
<code>evidence</code>	✓	Array of absolute URIs — include both <code>.pdf</code> and <code>.txt</code> versions
<code>outcome</code>	✓	The verifiable result of the intervention
<code>domain</code>		System category: Human, Civic, Physical, or Ecological
<code>description</code>		Narrative summary of the diagnosis and intervention

7. AI Integration — How It Works

AI systems with web access can autonomously traverse the full audit chain:

- **Step 1 — Load resume:** Fetch the resume HTML and parse all `data-kcm` attributes.
- **Step 2 — Load registry:** Fetch `/claims.json` and map claim IDs to evidence URIs.
- **Step 3 — Retrieve artifacts:** Fetch each evidence file. If the PDF fails, fall back to the matching `.txt` sidecar.
- **Step 4 — Verify integrity:** Compare SHA-256 hashes against `site_manifest.json.asc`.
- **Step 5 — Report verdict:** Return a claim-by-claim assessment: VERIFIED / CONFLICT / UNSTABLE.

AI Compatibility (March 2026):

- Grok (X.ai) — Fully compatible. Best-in-class for live forensic crawls.
- ChatGPT / Claude / Perplexity — Compatible with Text Bridge and CORS enabled.

8. Benefits

- **Verifiable credibility:** Claims are backed by primary artifacts, not self-assertion.
- **AI-readable by design:** The protocol is built for the era of AI-assisted hiring — your resume can be audited programmatically.
- **Transparency:** Evidence is publicly accessible with open CORS. Nothing is hidden behind a login.
- **Portable:** Your evidence lives on your own domain. No platform dependency.
- **Self-sovereign:** No issuer needed. You hold and serve your own proof.
- **Future-compatible:** Evidence hashes can be exported into verifiable credentials when wallets become mainstream.

9. Future-Proofing & Hybrid Vision

The Kerekes Handshake is an *interim bridge* — designed for the class of accomplishments no institution will ever certify: solo audits, civic interventions, DIY builds, local wins.

Once digital wallets and Verifiable Credentials (VCs) become mainstream, you'll be able to export your evidence hashes directly into them. The SHA-256 hashes in your `site_manifest.json.asc` are already in the right format for this transition. **Hybrid wins.**

Open Invitation:

This might be overkill. Wallets are probably coming and they're better for privacy. Tell me what sucks. Roast it. Fork it. Make it better.

Feedback > ego. — github.com/jeffreykerekeshandshake/kerekeshandshake

10. Technical Infrastructure (Advanced)

The core protocol requires only `claims.json`, KCM markup, and evidence files. A production-grade deployment for AI agent support adds:

10.1 Agent Discovery: `ai-plugin.json`

A standardized discovery file at `/.well-known/ai-plugin.json` provides a "front door" for autonomous AI agents. Key fields include `name_for_model`, `description_for_model`, and the `api` endpoint pointing to your OpenAPI spec.

10.2 The Schema: `openapi.yaml`

The `openapi.yaml` file is the machine-readable instruction manual for your Evidence Vault. It defines available endpoints (e.g., `/evidence/{filename}`, `/claims.json`) and expected data structures.

This allows an AI agent to programmatically request a specific document by `claim_id` without human intervention.

10.3 The Gatekeeper: `_worker.js` (CORS)

For Cloudflare Pages hosting, `_worker.js` manages cross-origin permissions. AI tools (ChatGPT, Grok) access your site from external domains — without the correct CORS header they will receive a Permission Denied error.

Required header for `/evidence/`:

```
Access-Control-Allow-Origin: *
```

The worker also ensures PDFs are served with the correct `Content-Type: application/pdf` header so agent parsers can read them correctly.

10.4 The Index: `llms-full.txt`

A consolidated plain-text index at `/llms-full.txt` provides a high-density "pre-flight" briefing for LLM agents. It contains: biographical context, identity disambiguation, the full archival index, and audit instructions. The agent reads this before deep-fetching individual artifacts, reducing unnecessary requests and improving extraction accuracy.

10.5 The "Buster" Philosophy

The query portal is a *Dare UI*. The underlying assumption is that the value of any professional claim is **zero** until verified. The protocol moves the burden of proof from the recruiter's intuition to the candidate's infrastructure. If you can't show it, you don't get credit for it.

11. Implementation Checklist (v1.5 Compliance)

An implementation is **v1.5 Compliant** when all six criteria are met:

- **Resume:** HTML tagged with `data-kcm` on all claim containers.
- **Text Bridge:** Every PDF in `/evidence/` has a matching `.txt` sidecar.
- **Registry:** `claims.json` maps every claim ID to at least one evidence URI.
- **Integrity Seal:** `site-manifest.json.asc` is PGP-signed and current.
- **Open Access:** CORS headers (`Access-Control-Allow-Origin: *`) enabled for `/evidence/`.
- **Audit Portal:** `/query/` page is live and serves the Forensic Manifesto prompt.

Advanced compliance also requires: `openapi.yaml` at site root, `ai-plugin.json` at `/.well-known/`, and `llms-full.txt` for LLM pre-flight context.

12. Defensive License — CC BY-SA 4.0

This protocol and all associated documentation are licensed under the **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

- **Attribution:** You must credit Jeffrey Kerekes as the original architect of the protocol.
- **ShareAlike:** If you build upon, transform, or create a new standard based on this protocol, you must distribute your contributions under the same license.
- **Purpose:** This ensures the Handshake remains a free, open utility and prevents proprietary platforms from privatizing the standard.

PGP Fingerprint: D39E 4ACE A4FE 3E6B 547F 58C4 6174 3446 DFA7 D48F

Verification: `curl -s https://jeffreykerekes.com/verify/pubkey.txt`

Conclusion

The Kerekes Handshake transforms resumes into verifiable systems. By linking professional claims to primary evidence through structured data, semantic markup, and cryptographic signing, it enables a new standard of credibility — one that works for the self-employed, the civic-minded, and anyone whose most important work will never appear on a diploma.

It is deliberately simple. It is deliberately open. It is designed to be forked, improved, and eventually superseded by something better.

Created by Jeffrey Kerekes — Systems Practitioner
jeffreykerekes.com | github.com/jeffreykerekes/kerekes-handshake

End Full Specification v1.5